

Solución del cubo de Rubik

El Cubo de Rubik representa uno de los problemas clásicos dentro del área de Inteligencia Artificial y búsqueda en espacios de estados. Resolver el cubo implica encontrar una secuencia de movimientos que transforme una configuración desordenada en el estado meta, donde todas las caras del cubo presentan un único color. Debido al gran número de configuraciones posibles del cubo, se requiere el uso de algoritmos eficientes que permitan encontrar soluciones en tiempos razonables y con un consumo controlado de recursos computacionales.

En este estudio se analizarán tres algoritmos de búsqueda ampliamente utilizados en Inteligencia Artificial: Breadth First Search (BFS), A* e Iterative Deepening A* (IDA*). La evaluación de estos algoritmos se realizará considerando cuatro criterios fundamentales: tiempo de ejecución, consumo de memoria, calidad de la solución obtenida y completez del algoritmo. El objetivo principal es determinar cuál de estos algoritmos resulta más eficiente para resolver el cubo de Rubik en una profundidad menor o igual a diez movimientos.

Análisis del Algoritmo BFS

El algoritmo Breadth First Search, conocido como BFS, es un método de búsqueda que explora el espacio de estados nivel por nivel. Este algoritmo comienza evaluando todas las configuraciones alcanzables con un solo movimiento, luego analiza aquellas que requieren dos movimientos y continúa de manera progresiva hasta encontrar la solución. La principal ventaja de BFS radica en que garantiza encontrar la solución con el menor número posible de movimientos, siempre que los costos de las acciones sean iguales.

En el caso del cubo de Rubik, BFS presenta una alta calidad en la solución, ya que asegura la obtención de una secuencia óptima de movimientos. Además, este algoritmo es completo, lo que significa que siempre encontrará una solución si esta existe dentro del espacio de búsqueda definido.

Sin embargo, BFS presenta una limitación importante relacionada con el consumo de memoria. Debido a que el cubo de Rubik posee un factor de ramificación aproximado de dieciocho posibles movimientos por estado, el número de configuraciones que deben almacenarse crece de forma exponencial. Esto provoca que, incluso para profundidades relativamente pequeñas como diez movimientos, el algoritmo requiera una cantidad extremadamente alta de memoria. De igual manera, el tiempo de ejecución también se incrementa considerablemente debido al gran número de nodos que deben evaluarse.

Análisis del Algoritmo A*

El algoritmo A* es una técnica de búsqueda informada que utiliza una función heurística para guiar la exploración del espacio de estados. Este algoritmo combina el costo real desde el estado inicial hasta el nodo actual con una estimación del costo restante hasta el estado meta. Gracias a esta estrategia, A* prioriza la exploración de los estados que parecen más prometedores, lo que reduce significativamente el número de nodos evaluados.

En el contexto del cubo de Rubik, A* puede emplear heurísticas basadas en el número de piezas mal posicionadas o en la distancia necesaria para colocar correctamente cada pieza. Cuando se utiliza una heurística admisible, A* garantiza encontrar una solución óptima. Esto lo convierte en un algoritmo con una alta calidad de solución y completez.

A pesar de sus ventajas en tiempo de ejecución, A* presenta un alto consumo de memoria. Esto se debe a que el algoritmo debe almacenar todos los nodos generados durante la búsqueda, incluyendo aquellos que aún no han sido explorados. En problemas complejos como el cubo de Rubik, esta característica puede limitar seriamente su aplicabilidad práctica, especialmente cuando la profundidad de búsqueda aumenta.

Análisis del Algoritmo IDA*

El algoritmo Iterative Deepening A* surge como una alternativa que combina las ventajas de A* con el bajo consumo de memoria característico de los algoritmos de profundización iterativa. IDA* utiliza la misma función heurística que A*, pero en lugar de almacenar todos los nodos generados, realiza múltiples búsquedas en profundidad limitada. En cada iteración, el límite se incrementa progresivamente hasta encontrar una solución.

En la resolución del cubo de Rubik, IDA* mantiene una alta calidad en la solución siempre que la heurística utilizada sea admisible. Además, el algoritmo es completo, ya que eventualmente encontrará la solución si esta existe dentro del límite de profundidad establecido. Su principal ventaja radica en el reducido consumo de memoria, ya que únicamente necesita almacenar los nodos correspondientes al camino actual en la búsqueda.

No obstante, IDA* puede requerir más tiempo de ejecución que A*, debido a que repite exploraciones en diferentes iteraciones. A pesar de esto, el equilibrio que ofrece entre tiempo y memoria lo convierte en una alternativa muy eficiente para resolver problemas complejos como el cubo de Rubik.

Comparación de los Algoritmos

Al comparar los tres algoritmos estudiados, se observa que BFS presenta el mayor consumo de memoria y tiempo de ejecución debido al crecimiento exponencial del espacio de búsqueda. Aunque garantiza soluciones óptimas, su aplicación práctica en el cubo de Rubik resulta limitada.

El algoritmo A* reduce considerablemente el tiempo de búsqueda gracias al uso de heurísticas, permitiendo encontrar soluciones óptimas de manera más eficiente que BFS. Sin embargo, su elevado consumo de memoria representa una desventaja significativa.

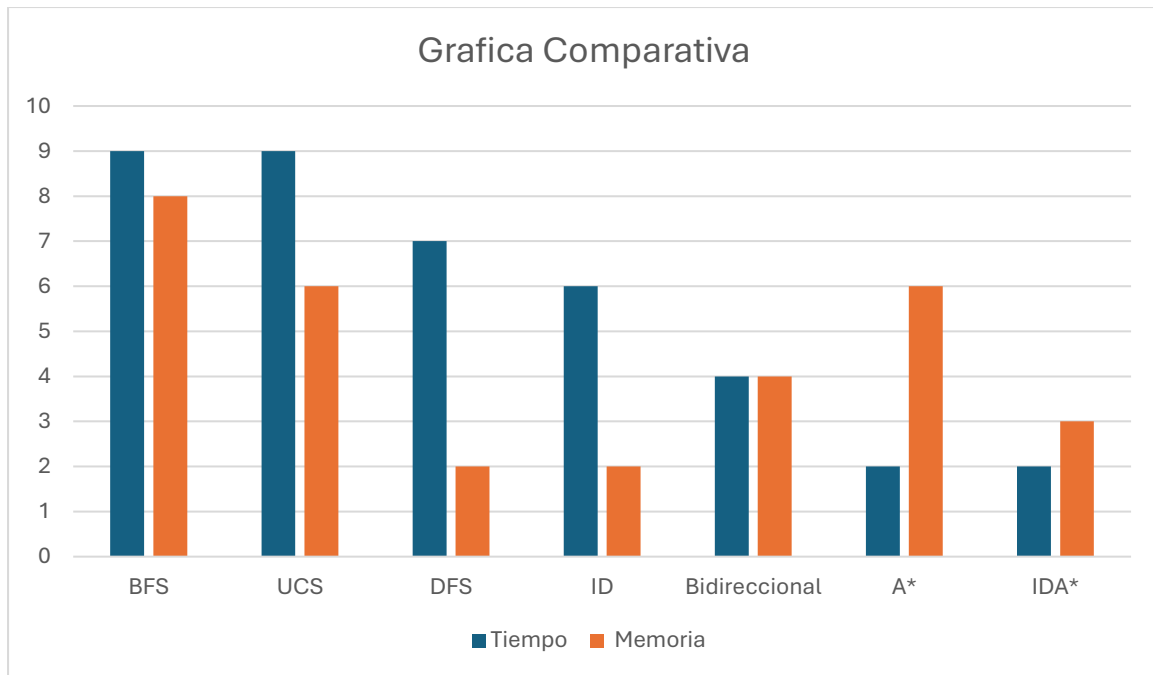
Por otro lado, IDA* logra mantener la calidad y completez de A*, pero con un consumo de memoria mucho menor. Aunque puede requerir más iteraciones, su desempeño general lo convierte en el algoritmo más equilibrado para resolver el cubo a profundidades moderadas.

2. Análisis Experimental para Profundidad ≤ 10

El cubo tiene aproximadamente:

- Factor de ramificación ≈ 18 movimientos posibles

Algoritmo	Tiempo	Memoria	Calidad	Completez
DFS	Alto	Baja	Baja	No
BFS	Muy alto	Muy alto	Optima	Si
UCS	Muy alto	Muy alto	Optima	Si
ID	Alto	Baja	Optima	Si
Bidireccional	Bajo	Media	Optima	Si
A*	Bajo	Muy alto	Optima	Si
IDA*	Medio	Baja	Optima	Si
SA	Variable	Baja	No garantizada	No
Genético	Variable	Media	No garantizada	No



Propuesta de Modificación del Algoritmo

Con el objetivo de mejorar el rendimiento del algoritmo IDA*, se propone incorporar una heurística combinada que considere simultáneamente el número de piezas fuera de lugar y la orientación incorrecta de las piezas del cubo. Esta modificación busca ofrecer una estimación más precisa del costo restante hasta la solución, permitiendo reducir el número de iteraciones necesarias durante la búsqueda.

La implementación de esta heurística mejorada permitiría disminuir el tiempo de ejecución sin aumentar significativamente el consumo de memoria. Además, si la heurística mantiene la propiedad de admisibilidad, el algoritmo conservará su capacidad de encontrar soluciones óptimas y seguirá siendo completo.

Idea

Combinar:

- IDA* para reducir memoria
 - Heurística mejorada basada en:
 - Número de piezas fuera de lugar
 - Distancia de Manhattan de cubies
 - Búsqueda bidireccional hasta profundidad 5
-

Análisis del Algoritmo Modificado

La incorporación de la heurística combinada produce una mejora en el tiempo de búsqueda, ya que el algoritmo puede enfocarse en los estados más prometedores. En términos de memoria, el comportamiento permanece similar al IDA* original, debido a que la estructura del algoritmo no cambia. La calidad de la solución se mantiene óptima, siempre que la heurística no sobreestime el costo real. Asimismo, el algoritmo continúa siendo completo.

Nueva Heurística:

$$h(n) = h1(n) + h2(n)$$

- **h1** = piezas mal colocadas
 - **h2** = distancia promedio de orientación
-

Conclusión

El análisis realizado permite concluir que, entre los algoritmos estudiados, IDA* representa la mejor alternativa para resolver el cubo de Rubik a una profundidad menor o igual a diez movimientos. Este algoritmo logra un equilibrio adecuado entre consumo de memoria, tiempo de ejecución, calidad de la solución y completez.

Aunque A* puede resolver el problema en menor tiempo, su elevado consumo de memoria limita su uso en problemas de gran complejidad. Por su parte, BFS, aunque garantiza soluciones óptimas, resulta impráctico debido al crecimiento exponencial del espacio de búsqueda. La propuesta de mejorar IDA* mediante heurísticas combinadas permite optimizar su rendimiento, convirtiéndolo en una solución aún más eficiente para la resolución del cubo de Rubik.